Realcryptik: Autonomous Election Verification with Zeeperio

Aikam Malhotra[1[]], Aleksander Essex[1[0000−0002−0228−0371]], and Jeremy Clark[2[0000−0002−3533−5965]]

[1] Western University, London, Canada
{amalho26,aessex}@uwo.ca
[2] Concordia University, Montreal, Canada
j.clark@concordia.ca

Abstract. The size of a Zeeperio proof is constant in the number of ballots and voters in an election, as is the time to verify it. It is compact enough for autonomous verification by a smart contract on a blockchain, such as Ethereum.

Keywords: end-to-end verifiable voting, zero-knowledge proofs, zk-SNARKs, polynomial interactive oracle proofs, smart contracts

1 Introduction

Decades of research on end-to-end verifiable (E2E-V) elections have shown strong cryptographic guarantees can be realized for protecting the integrity of elections and the secrecy of voter ballots [16]. In more recent years, pilot projects and deployments of E2E-V platforms have also demonstrated that these systems are usable and scalable to an appropriate degree [2, 8, 4, 12]. Despite these efforts, the vendors of systems for governmental elections continue to run unverifiable elections where the content of individual ballots can be determined by at least one trusted entity in the platform.

In this paper, we contribute to a broader discussion the cryptographic voting community has been having for some number of years: do we compromise our ideals and offer watered down systems that the industry might find more palatable if they offer tangible security advantages over the status quo? We relate this to the term Realpolitik, coined in 19th century Germany to describe statecraft based on pragmatic calculations rather than ideology, and propose *Realcryptik* to describe such a pragmatic approach to election verification.

Flavours of Realcryptik already permeate the research literature. Despite opposition to internet voting, Helios [1] is perhaps the most used E2E-V voting technology. While researchers argue coercion resistance is essential for internet voting, Helios provides none. Scantegrity II [9] was the first E2E-V system to see governmental use. It is built around the compromise that in a paper ballot system, it is ok for optical scanner to learn how the voter voted, side-stepping the complex ballot mechanics its predecessors to avoid this. In recent years, serious

discussions have arose around whether E2E-V should be largely abandoned in favour of risk limiting audits? Should we affirm the use of ballot marking devices when we know most voters do not carefully check their output?

We propose a different compromise in this paper: let EAs and their vendors see the raw ballots, while protecting them from all outsiders (receipt-freeness). For the EA, we still rely on legal protections implemented through procedures but not offer cryptographic protection. Should add more arguments...

The research question of this paper is how to modify the cryptography of an E2E-V voting system to provide integrity, without the need to maintain ballot secrecy from the EA (but still provide receipt-freeness otherwise). The simpler threat model means we can streamline the cryptography, in particular we can prove that encrypted (or committed in our case) ballots are tallied correctly rather than computing the tally under encryption. This shifts the protocol away from multiparty computation and threshold cryptography to essentially an all encompassing zero knowledge proof (ZKP). While ZKPs have been used in voting systems from almost their invention in 1980s and onward, we focus on modern succinct ZKPs called zk-SNARKs.

Zeeperio. We apply this modification to an existing E2E-V back-end called Eperio [13]. Eperio is a simplification of Scantegrity II's backend (known as a switchboard) and is design to be simple enough that a non-expert validator can audit the election using a spreadsheet, such as MS Excel. Eperio's security was studied in detail using formal methods at CCS 2021 paper [3]. By replacing Eperio's cut-and-choose based proof with a zk-SNARK, it obviously is no longer verifiable by hand. However this is a deliberate choice. Traditionally voters would either have to verify an election themselves or rely on a trusted entity (political party, watchdog) to perform this verification—hence the need to simplify elections to a spreadsheet. Today we have systems like Ethereum that provide a peer-to-peer network that is effectively validating that computational outputs of arbitrary code, called smart contracts, are correct. Ethereum protects assets with a market capitalization of $100+B USD and it can protect election results too, as long as the results are verifiable within its very restricted limits (known as gas consumption). If the verification task can be reduced to a succinct (zk-SNARK) proof, voters do not need check the proof at all.

Our contribution is designing a custom succinct argument for Eperio that we call Zeeprio. Our protocol uses polynomial interactive oracle proofs (PIOPs), which have been used to build zk-SNARKs for any arithmetic circuit (and thus any computation), the best known system being Plonk [14]. The size of a Zeeperio proof is constant in the number of ballots and voters in an election, as is the time to verify it. It is compact enough for autonomous verification by a smart contract on a blockchain, such as Ethereum. Zeeperio proofs are linear (as opposed to constant) in the number of candidates, but this seems acceptable as reading the final tally or checking an audited ballot is linear in the number of candidates.

2 Background and Related Work

2.1 Eperio

We assume the reader is already familiar with end-to-end verifiable (E2E-V) voting protocols which allow for: (i) an election authority to prove an election tally is computed correctly without learning individual voter choices, and for (ii) voters to check their individual ballots are included unmodified in the tally without being able to reveal how they voted to anyone else. Eperio is an E2E-V tallying protocol (or 'backend') that composes with several styles of paper ballots, including Pret a Voter and Scantegrity I. A brief description of Eperio is provided in Figure **??** with a Pret a Voter style ballot.

In addition to succinctness, we also clean up a few issues that were not fully resolved in Scantegrity II and Eperio:

- As pointed about by Basin *et al.* [3], Eperio does not prove ballots were not overvoted; Zeeperio does (§3.7).
- As acknowledged by Essex *et al.* [13], Eperio does not support Scantegrity II ballots; Zeeperio does (§4.2).
- As acknowledged by Chaum *et al.* [9, 10], Scantegrity II supports dispute resolution but it requires all confirmation codes on the ballot to be revealed. Zeeprio offers a simple proof of incorrect code without revealing information about all codes (§4.3).

A remaining issue, pointed out by Basin et al [3], are clash attacks [17] which are not explicitly addressed by Eperio. However they are applicable to all major E2E-V voting systems and are addressed through strict polling place policies that denote which ballot is given to which voter.

2.2 Succinct Proofs

While zero knowledge proofs/arguments (ZKPs) date back to the 1980s [15, 6], classic ZKPs produce proofs that are at least linear in the size of the statement being proven. By contrast, succinct zero knowledge arguments (SNARKs) are sublinear, typically with a logarithmic or constant proof size. A Polynomial Interactive Oracle Proof (PIOPs) is a pattern that a number of SNARKs have in common [11, 7]. In this pattern, the prover specifies polynomials and verifiers challenge the prover to open these polynomials at random points. PIOP does not say *how* to make a SNARK, it is just a general pattern.

To realize a PIOP, the prover must send a succinct representation of each polynomial from the prover to the verifier, which commits the prover to those polynomials. When opening a polynomial, the verifier must be certain it is the opening of the same polynomial that was committed to. PIOPs rely on a cryptographic primitive called a polynomial commitment scheme (PCS) to handle this. The rest of a PIOP protocol does not use any cryptography. A number of PCSs have been proposed, the most prominent are KZG (very short, based on

bilinear pairings, requires a trusted setup) and (DEEP-)FRI (longer, based on hash functions, conjectured to be post-quantum secure, no trusted setup). In Zeeperio, we use KZG because it integrates well with Ethereum. Specifically we use the Pedersen variant of KZG with privacy and we ...

KZG uses a universal trusted setup: a secret value τ is chosen at random, used to compute a structured reference string (SRS) $\left\langle g^{\tau}, g^{\tau^2}, g^{\tau^3}, \ldots \right\rangle$ in a pairing-friendly group defined by generator g, and then erased. Knowledge of τ after the setup is sufficient to fake KZG openings and thus forge false ZKP transcripts. However the setup can be run as an open participation multiparty computation where participants prove their contribution is correct and only one honest participant must forget τ for the SRS to be secure. Ethereum ran an open trusted setup in 2023, accepting 100K contributions over 208 days. A single SRS, such as this, can be shared by everyone wanting to do a KZG commitment (known a universal trusted setup).

In practice, zk-SNARKs for a specific computation can be deployed in different ways: (a) written in a high-level language and compiled to VM instructions, (b) written in special markup languages and compiled into arithmetic circuits, or (c) written as a custom arithmetic circuit. In each case, these representations are fed into a general purpose proof system for circuits or zkVMs. Each level of abstraction adds overhead and costs to the prover. Our approach is 'closer to the metal.' We do not use circuits or Plonk itself. We instead go underneath what Plonk provides and use the same kind of building blocks it uses to make a custom PIOP argument for Zeeperio.

2.3 Smart Contracts

Smart contracts are programs deployed on a blockchain, such as Ethereum. They only execute when a user calls a function. The blockchain's consensus enforces the function call executes correctly and the state of the code is updated accordingly. In Zeeprio, a smart contract will accept an election proof and run the verification of it, reporting if it is correct or incorrect. Since smart contracts can implement arbitrary code, they can support cryptographic operations however this approach is usually prohibitively expensive. Ethereum offers cheaper 'pre-compiles' for pairing-based operations on the elliptic curve `alt-bn128`. On Ethereum's testnet, support for curve `bls12-381` has been staged for future deployment, as well as KZG-specific operations over `bls12-381`, using the forementioned SRS. Zeeperio on `alt-bn128` or `bls12-381`

3 Zeeperio

Parameters. Let $\mathcal{B}$ be the number of printed ballots and $\mathcal{C}$ the number of candidates per ballot. Let $\mathcal{P} = \mathcal{BC}$ be the number of ballot positions. Let n be the padded column length (a power of two) with $n \geq \mathcal{P}$ (the need is explained below). Let $\mathbb{F}_q$ be a finite field of size q. A ballot has ID $b \in \{0, \ldots, \mathcal{B}-1\}$ and occupies a *block*: $I_b = \{\, b\mathcal{C},\, b\mathcal{C}+1,\, \ldots,\, b\mathcal{C}+(\mathcal{C}-1) \,\} \subseteq \{0, \ldots, n-1\}$.

Walkthrough of a constraint. Notice from Figure **??**, the mark column can only contain a 0 or 1. This is one small fact about what constitutes a valid election table. With ZKPs, we call this a *constraint*. Zeeperio consists of 16 similar constraints that all must hold in any valid election table. Here we will demonstrate a complete argument for this single constraint (which we denote as c2.2 later) to demonstrate how a PIOP works. This constraint is adapted from [5].

The mark column can be considered a one-dimensional vector or array $A_{\mathsf{mark}}[i]$ of length $\mathcal{P}$. Elements in the array will be encoded as elements of $\mathbb{F}_q$, the exponent field of a pairing-friendly elliptic curve such as `alt-bn128` or `bls12-381`. Cryptographically speaking, the array $A_{\mathsf{mark}}[i]$ could thus hold any integer in $[0, q-1]$ but the constraint says it is only a valid election table if it is further restricted to $[0, 1]$.

To prove the constraint, the prover (election authority) will send to the verifier (smart contract) a succinct (one group element) commitment (hiding and binding) to the entire mark column $A_{\mathsf{mark}}[i]$. While vector commitments exist, in a PIOP, the prover commits to $A_{\mathsf{mark}}[i]$ indirectly by first encoding the data from $A_{\mathsf{mark}}[i]$ into a univariate polynomial over $\mathbb{F}_q$ and then commits to this polynomial representation of $A_{\mathsf{mark}}[i]$ using a polynomial commitment scheme such as KZG.

There a variety of ways to encode vectors into polynomials (we refer interested reader to plonkbook), we will focus on the method we will use. The prover chooses a set of x-coordinates, such as $\{0, 1, 2, 3, \ldots\}$ which is called an evaluation domain $\mathcal{H}$. It then takes the data from $A_{\mathsf{mark}}[i]$ and forms a set of points $\{(0, A_{\mathsf{mark}}[0]), (1, A_{\mathsf{mark}}[1]), \ldots\}$. The prover interpolates a univariate polynomial through these points: $\mathsf{Mark}(X)$. It commits to $\mathsf{Mark}(X)$ using the polynomial commitment scheme, and sends the commitment to the verifier.

Interpolation is expensive for the prover so provers use an optimization. The evaluation domain is arbitrary and can be anything as long as the prover and verifier agree. Let $\omega \in \mathbb{F}_q$ generate a multiplicative subgroup of size n. Then set $H = \{\omega^0, \omega^1, \ldots, \omega^{n-1}\} \subseteq \mathbb{F}_q$. Doing so offers three advantages and one drawback. Interpolating a polynomial over this domain can be accomplished in $\mathcal{O}(n \log n)$ instead of $\mathcal{O}(n^2)$. Traversing the domain easy: the next index is the current index multiplied by ω and because it is closed under multiplication, the 'next' element after the last element ω^{n-1} wraps back to the first element $\omega^{n-1} \cdot \omega = \omega^n = \omega^0$. Finally, a polynomial that is 0 at each $X \in H$ has a simple closed form: $\mathsf{Z}_H(X) = X^n - 1$. The drawback is that the prover cannot use arrays of arbitrary length $\mathcal{P}$ and expect that an order n subgroup will just exist. The subgroups are based on the parameters of the elliptic curve and curves like `alt-bn128` and `bls12-381` are designed to offer subgroups of sizes $\{2, 2^2, 2^3, 2^4, \ldots\}$ up to 2^{12} for `alt-bn128` and 2^{32} for `bls12-381`.

In Zeeprio, if we have a column of length $\mathcal{P}$, we will zero pad it the nearest power of 2. Thus n is the padded column length (a power of two) with $n \geq \mathcal{P}$. For this example constraint (mark column is binary), the presence of zero padding does not impact the constraint. However for other constraints, we will have to explicitly handle the padding separately from the election table data.

At this point, the verifier has a polynomial commitment to $\mathsf{Mark}(X)$ which we will call K_{Mark}. How does the prover demonstrate it is binary? The prover will construct another polynomial as follows: $V(X) := \mathsf{Mark}(X)\big(\mathsf{Mark}(X) - 1\big)$. It will commit to $V(X)$ and send K_V to the verifier. If $\mathsf{Mark}(X)$ is binary for all $X \in H$, then $V(X)$ will be zero for all $X \in H$. If $\mathsf{Mark}(X)$ is not binary for at least one $X \in H$, $V(X)$ will be non-zero at the same X. We have converted the constraint $\mathsf{Mark}(X)$ is binary into:

1. $V(X)$ is correctly constructed from $\mathsf{Mark}(X)$
2. $V(X)$ it is zero for all $X \in H$. A polynomial that is is zero for all $X \in H$ is called a *vanishing polynomial* (on H).

To prove that $V(X)$ is correctly constructed from $\mathsf{Mark}(X)$, the verifier has K_{Mark} and K_V. The verifier will (or the prover using the strong Fiat-Shamir heuristic) choose a random evaluation point $\zeta \notin H$. Using the polynomial commitment scheme, the prover will open the polynomials at the value ζ sending $V(\zeta)$, $\mathsf{Mark}(\zeta)$, and opening proofs for each to the verifier. The verifier will check the opening proofs and check that $V(\zeta) := \mathsf{Mark}(\zeta)\big(\mathsf{Mark}(\zeta) - 1\big)$. Just because the relation holds at the point ζ, what does that say about it holding at each $X \in H$? The answer is that it overwhelmingly holds due to a lemma named Schwartz-Zippel that we will not review further here.

Next the prover will demonstrate that $V(X)$ vanishes on H. Recall that $\mathsf{Z}_H(X) = X^n - 1$ is a vanishing polynomial on H. The prover will construct another polynomial $Q(X) := V(X)/\mathsf{Z}_H(X)$, commits to $Q(X)$, and sends the commitment K_Q to the verifier. If and only if $V(X)$ is zero at every point of H, $\mathsf{Z}_H(X)$ exactly divides $V(X)$ and results in a proper polynomial $Q(X)$, otherwise $Q(X)$ will be a rational function not reducible to a polynomial. $Q(X) := V(X)/\mathsf{Z}_H(X)$ can be rearranged as $V(X) - Q(X) \cdot (X^n - 1) = 0$. Using ζ from above, the prover sends $Q(\zeta)$ and its opening proof. The verifier checks $V(\zeta) - Q(\zeta) \cdot (\zeta^n - 1) = 0$. By Schwartz–Zippel, if this equality holds at a uniformly random $\zeta \notin H$, then except with negligible probability it holds as a polynomial identity, which implies that $V(X)$ vanishes on H. □

3.1 Zeeperio Constraints

Zeeperio is made up of 16 constraints. Each constraint follows the similar pattern to the example constraint: a set of polynomials are committed to such that a relationship between them will be a vanishing polynomial if and only if the constraint is true. Once the constraint is converted into a vanishing polynomial, proving the vanishing polynomial is constructed correctly and that it actually vanishes is identical to the process used above.

In fact, given a set of vanishing polynomials $\{V_1(X), V_2(X), \ldots, V_m(X)\}$ and their corrosponding quotiant polynomials, the check can be batched into a single check. The prover will form a 'super-polynomial' $\mathsf{Batch}(X)$ where each coefficient is one of the vanishing polynomials:

Table 1. A set of polynomials that can be computed before the beginning of the election. $\mathsf{Sel}_b(X)$ is the one exception, it will not be precomputed but rather computed during a receipt check just for the one ballot b that the voter wants to check.

Description	**Polynomial**
Polynomial that is zero on H	$\mathsf{Z}_H(X) = X^n - 1$
Mask that is 0 on all of H except possibly at $X = \omega^0$	$\mathsf{Mask}_0(X) = \dfrac{\mathsf{Z}_H(X)}{X - \omega^0}$
Mask that is 0 on all of H except possibly at $X = \omega^{n-1}$	$\mathsf{Mask}_{n-1}(X) = \dfrac{\mathsf{Z}_H(X)}{X - \omega^{n-1}}$
Polynomial that is 0 up to padding $[0, \mathcal{P}-1]$	$\mathsf{Z}_{\mathsf{head}}(X) = \prod_{i=0}^{\mathcal{P}-1} \left(X - \omega^i\right)$
Polynomial that is 0 for the last $\mathcal{C}$ rows (zeros at $\omega^{n-1}, \ldots, \omega^{n-\mathcal{C}}$)	$\mathsf{Mask}_{\mathsf{tail}}(X) = \prod_{t=0}^{\mathcal{C}-1} \left(X - \omega^{n-1-t}\right)$
Lagrange basis over H used below	$L_i(X) = \prod_{\substack{0 \le j < n \\ j \ne i}} \dfrac{X - \omega^j}{\omega^i - \omega^j}$
Polynomial with a 1 at the start of every ballot (indices $i \equiv 0 \pmod{\mathcal{C}}$ up to padding) and 0 elsewhere	$\mathsf{Sel}_{\mathsf{blk}}(X) = \sum_{j=0}^{\lfloor n/\mathcal{C} \rfloor - 1} L_{j\mathcal{C}}(X)$
Polynomial that is 1 for all rows belonging to ballot-b ($I_b = \{b\mathcal{C}, \ldots, b\mathcal{C} + \mathcal{C} - 1\}$) and 0 elsewhere	$\mathsf{Sel}_b(X) = \sum_{k=0}^{\mathcal{C}-1} L_{b\mathcal{C}+k}(X)$

$$\mathsf{Batch}(Y) = \sum_{i=1}^{m} Y^i \Big(V_i(X) - Q_i(X)\, \mathsf{Z}_H(X) \Big)$$

A polynomial where every coefficient is 0 is a zero-polynomial. Therefore the verifier (or prover via strong Fiat-Shamir) will choose a random $\zeta \in \mathbb{F}_q \notin H$ as before and set $X = \zeta$, which overwhelmingly makes every coefficient zero. To see if every coefficient is zero, it will also choose a random $\alpha \in \mathbb{F}_q$ and set $Y = \alpha$. If every $V_i(X)$ vanishes, then $\mathsf{Batch}(Y)$ is a zero polynomial and:

$$\mathsf{Batch}(\alpha) = \sum_{i=1}^{m} \alpha^i \Big(V_i(\zeta) - Q_i(\zeta)\, \mathsf{Z}_H(\zeta) \Big) = 0.$$

3.2 Public Polynomials

Zeeperio uses several public helper polynomials that depend only on the election 'shape' (e.g., n, $\mathcal{B}$, $\mathcal{C}$, and hence $\mathcal{P} = \mathcal{BC}$), and not on any voter choices or on the contents of the committed columns. They are enumerated in Table 1. Some of these helpers are linear or superlinear in n to construct (for example, explicit products or explicit sums of Lagrange basis polynomials). This does not undermine the succinctness claims of Zeeprio: the pre-computation is offline, public,

and independent of the witness. In systems like Plonk, these types of polynomials are considered as a 'verifier's key.' In practice, election administrators can standardize on a small set of template sizes and these polynomials can be precomputed once for each election template. If an election uses fewer than the template's maximum number of ballots or candidates, the remaining rows are constrained to be empty: extra ballots are unused and extra candidates receive zero votes, which is directly reflected in the published tally.

3.3 Election Specific Polynomials

The Eperio tables for an election consist of three (padded) columns which will be encoded into polynomials evaluated over H. We call the polynomials $\mathsf{Code}(X)$, $\mathsf{Audit}(X)$, and $\mathsf{Mark}(X)$. How each index maps to a unique ballot ID and candidate is provided as a public template and columns will be sorted by ballot ID and then candidate (see Figure **??**). The final tally is a vector $\mathsf{Tally}[0..\mathcal{C}-1] \in \mathbb{F}_q^{\mathcal{C}}$.

Across all the constraints, we will use an additional 5 'helper' polynomials. We will introduce them as needed, however they consist of two (additive, suffix) accumulator polynomials: $\mathsf{Acc}_{\mathsf{Audit}}(X)$ and $\mathsf{Acc}_{\mathsf{Mark}}(X)$, where $\mathsf{Acc}_{\mathsf{Audit}}(\omega^i) = \sum_{t=i}^{n-1} \mathsf{Audit}(\omega^t)$; likewise for $\mathsf{Mark}(X)$. We also define a 'stridded' accumulator $\mathsf{Acc}_{\mathsf{Vote}}(X)$ that we will describe with constraints 4. Finally we also define two block sum helper polynomials: $\mathsf{BlkSum}_{\mathsf{Audit}}(X) = \sum_{k=0}^{\mathcal{C}-1} \mathsf{Audit}(\omega^k X)$ and $\mathsf{BlkSum}_{\mathsf{Mark}}(X) = \sum_{k=0}^{\mathcal{C}-1} \mathsf{Mark}(\omega^k X)$.

3.4 Audit Constraints

Due to limited space in the paper, we cannot fully explain each of the 16 constraints. Instead we provide some intuition for them.

$$
\begin{aligned}
V_{c1.1}(X) &:= \mathsf{Audit}(X)\,\mathsf{Z}_{\mathsf{head}}(X) \\
V_{c1.2}(X) &:= \mathsf{Audit}(X)\big(\mathsf{Audit}(X) - 1\big) \\
V_{c1.3}(X) &:= \big(\mathsf{Sel}_{\mathsf{blk}}(X)\mathsf{BlkSum}_{\mathsf{Audit}}(X)\big)\big(\mathsf{Sel}_{\mathsf{blk}}(X)\mathsf{BlkSum}_{\mathsf{Audit}}(X) - \mathcal{C}\big) \\
V_{c1.4}(X) &:= \big(\mathsf{Acc}_{\mathsf{Audit}}(X) - \mathsf{Audit}(X) + \mathsf{Acc}_{\mathsf{Audit}}(\omega X)\big) \cdot (X - \omega^{n-1}) \\
V_{c1.5}(X) &:= \big(\mathsf{Acc}_{\mathsf{Audit}}(X) - \mathsf{Audit}(X)\big) \cdot \mathsf{Mask}_{n-1}(X) \\
V_{c1.6}(X) &:= \big(\mathsf{Acc}_{\mathsf{Audit}}(X) - \mathsf{Sum}_{\mathsf{Audit}\times\mathcal{C}}\big) \cdot \mathsf{Mask}_0(X)
\end{aligned}
$$

The audit column is well-defined for an election if all padding bits have the value 0 (c1.1) and the column is otherwise binary (c1.2). Each ballot spans $\mathcal{C}$ positions, so the audit column should contain a contiguous block of $\mathcal{C}$ ones for an audited ballot, and a block of $\mathcal{C}$ zeros for unaudited ballots. The mechanism is to compute a block-sum over each ballot, $\mathsf{BlkSum}_{\mathsf{Audit}}(X) = \sum_{k=0}^{\mathcal{C}-1} \mathsf{Audit}(\omega^k X)$, then restrict attention to ballot-start indices using $\mathsf{Sel}_{\mathsf{blk}}(X)$. For a well-formed audit column, the filtered block-sum is always either 0 (not audited) or $\mathcal{C}$ (audited), and never an intermediate value. Note that after choosing ζ, in order to

check $\mathsf{BlkSum}_{\mathsf{Audit}}(X)$ is well-formed, the verifier requires $\mathcal{C}$ opened points from $\mathsf{Audit}(\omega^0\zeta)$ to $\mathsf{Audit}(\omega^{\mathcal{C}-1}\zeta)$, so this check is linear in $\mathcal{C}$ (but more importantly constant in $\mathcal{P}$ and n).

The EA will assert the number of print audits in the election as the integer $\mathsf{Sum}_{\mathsf{A}}$. Since each print audit ballot contains a block of $\mathcal{C}$ 1's, the number of 1's in $A(X)$ must be $\mathsf{Sum}_{\mathsf{A}\times\mathcal{C}} = \mathsf{Sum}_{\mathsf{A}} \cdot \mathcal{C}$. To reason about the sum across a polynomial on H, many protocols use accumulator polynomials [14, 5]. $\mathsf{Acc}_{\mathsf{Audit}}(X)$ is intended to hold a running suffix-sum of the audit column down the domain order (including padding, which is forced to be 0 by c1.1):

$$\begin{aligned}\mathsf{Acc}_{\mathsf{Audit}}(\omega^i) &:= \mathsf{Audit}(\omega^i) + \mathsf{Acc}_{\mathsf{Audit}}(\omega^{i+1}) \quad \text{for } i = 0, 1, \ldots, n-2,\\ \mathsf{Acc}_{\mathsf{Audit}}(\omega^{n-1}) &:= \mathsf{Audit}(\omega^{n-1}).\end{aligned}$$

Constraint c1.4 and c1.5 check each line of this recurrance respectively, guarenting that $\mathsf{Acc}_{\mathsf{Audit}}(X)$ is formed correctly. The first value in $\mathsf{Acc}_{\mathsf{Audit}}(X)$ holds the sum, so c1.6 ensures this value is equal to $\mathsf{Sum}_{\mathsf{A}\times\mathcal{C}}$.

3.5 Mark Constraints

$$\begin{aligned}
V_{c2.1}(X) &:= \mathsf{Mark}(X)\,\mathsf{Z}_{\mathsf{head}}(X)\\
V_{c2.2}(X) &:= \mathsf{Mark}(X)\big(\mathsf{Mark}(X) - 1\big)\\
V_{c2.3}(X) &:= \big(\mathsf{Sel}_{\mathsf{blk}}(X)\mathsf{BlkSum}_{\mathsf{Mark}}(X)\big)\big(\mathsf{Sel}_{\mathsf{blk}}(X)\mathsf{BlkSum}_{\mathsf{Mark}}(X) - 1\big)\\
V_{c2.4}(X) &:= \big(\mathsf{Acc}_{\mathsf{Mark}}(X) - \mathsf{Mark}(X) + \mathsf{Acc}_{\mathsf{Mark}}(\omega X)\big) \cdot (X - \omega^{n-1})\\
V_{c2.5}(X) &:= \big(\mathsf{Acc}_{\mathsf{Mark}}(X) - \mathsf{Mark}(X)\big) \cdot \mathsf{Mask}_{n-1}(X)\\
V_{c2.6}(X) &:= \big(\mathsf{Acc}_{\mathsf{Mark}}(X) - \mathsf{Sum}_{\mathsf{Mark}}\big) \cdot \mathsf{Mask}_0(X)
\end{aligned}$$

The constraints on the mark column parallel the constraints on the audit column. Padding is zero (c2.1), the column is binary (c2.2), and the total number of votes match the sum announced by the election authority $\mathsf{Sum}_{\mathsf{Mark}}$ (c2.4–c2.5). The main difference is constraint c2.3. For each ballot in the audit column (previous section), the ballot's block is either all 0 or all 1, summing to either 0 or $\mathcal{C}$. For the vote column (this section), each ballot block is either all 0 (not voted) or contains a single 1 (voted). Since it is binary (c2.2), the EA can demonstrate the sum of each ballot block is either 0 or 1 which is c2.3.

3.6 Cross-Column Constraints

$$V_{c3.1}(X) := \mathsf{Audit}(X)\,\mathsf{Mark}(X)$$

A final constraint demonstrates that no ballot is both voted and audited. This is simple: an unvoted ballot will have all 0's in its ballot block, while an

unaudited ballot will have the same. Therefore the Hadamard product (element-wise multiplication) of the audit column with the mark column will be 0 in every element if it is properly formed. In the malicious case, if a ballot is both voted and audited, at least one element of the ballot block will contain a 1 in both columns and the product of the columns will not be 0 at that element, thus $V_{c3.1}(X)$ will not vanish.

3.7 Tally Constraints

$$
\begin{aligned}
V_{c4.1}(X) &:= \big(\mathsf{Acc}_{\mathsf{Vote}}(X) - \mathsf{Mark}(X) + \mathsf{Acc}_{\mathsf{Vote}}(\omega^{\mathcal{C}} X)\big) \cdot \mathsf{Mask}_{\mathsf{tail}}(X) \\
V_{c4.2}(X) &:= \big(\mathsf{Acc}_{\mathsf{Vote}}(X) - \mathsf{Mark}(X)\big) \cdot \frac{\mathsf{Z}_H(X)}{\mathsf{Mask}_{\mathsf{tail}}(X)} \\
V_{c4.3,j}(X) &:= \big(\mathsf{Acc}_{\mathsf{Vote}}(X) - \mathsf{Tally}[j]\big) \cdot \frac{\mathsf{Z}_H(X)}{X - \omega^j}
\end{aligned}
$$

The EA will announce the tally as a list of vote totals for each of the $\mathcal{C}$ candidates: $\mathsf{Tally}[j]$ for $j \in \{0, \ldots, \mathcal{C}-1\}$. The marks for candidate i will be in every i-th element of $\mathsf{Mark}(X)$ over H. The EA will build an additive, suffix accumulator $\mathsf{Acc}_{\mathsf{Vote}}(X)$ however it will be *strided*:

$$
\mathsf{Acc}_{\mathsf{Vote}}(\omega^i) := \sum_{t=0}^{\lfloor \frac{n-1-i}{\mathcal{C}} \rfloor} \mathsf{Mark}\big(\omega^{i+t\mathcal{C}}\big), \qquad i \in \{0, \ldots, n-1\}.
$$

The result is that $\mathsf{AccVote}(\omega^c)$ is equal the total number of marks appearing at indices congruent to $c \bmod \mathcal{C}$ across all ballots (and padding). Note there is no assumption that $c|n$, the sum for some candidates may be one element longer than others. Constraint c4.1 demonstrates the accumulator is built correctly, assuming it starts with the right values. Constraint c4.2 demonstrates the starting values are correct. Constraint c4.3 demonstrates that the published tally matches the sum of each candidate's vote. Constraint c4.3 is run $\mathcal{C}$ times, once for each candidate's total votes which will be contained in the first $\mathcal{C}$ elements of $\mathsf{AccVote}(X)$ on H.

4 Voter Checks

Eperio, like many voting systems, split the E2E-V proof into two parts: a universal verifiable proof that the tally is correct, and a voter verifiable proof that receipts are correct. All the constraints from Section 3 correspond to the former proof, while in this section we describe the voter verifiable checks.

4.1 Print Audit

To support print-audit verification, we use a ballot-specific *block mask* polynomial that isolates the $\mathcal{C}$ rows corresponding to a particular ballot. For a ballot

ID b, let $\mathsf{Mask}^{(b)}_{\mathsf{ballot}}(X)$ be the publicly checkable polynomial defined over the domain H by

$$\mathsf{Mask}^{(b)}_{\mathsf{ballot}}(\omega^i) = \begin{cases} 1, & i \in I_b, \\ 0, & \text{otherwise}, \end{cases} \qquad i \in \{0, \ldots, n-1\},$$

i.e., it is the interpolation of a 0/1 array that is 1 exactly on the ballot block $I_b = \{b\mathcal{C}, \ldots, b\mathcal{C} + (\mathcal{C} - 1)\}$. Given the ballot ID b, the election authority (EA) proves the print-audit data for that ballot by opening the committed column polynomials at each of the $\mathcal{C}$ domain points in the block, returning the evaluations $\{\mathsf{Audit}(\omega^i)\}_{i \in I_b}$, $\{\mathsf{Mark}(\omega^i)\}_{i \in I_b}$, and $\{\mathsf{Code}(\omega^i)\}_{i \in I_b}$ together with a single batched opening proof for these evaluations.

4.2 Receipt Check

To support receipt checking without revealing which candidate-position within a ballot was marked, the EA uses a shuffle-based protocol that hides the canonical ordering behind a secret permutation. First, the EA defines a public *ballot-ID column* $\mathsf{BallotID}(X)$ that encodes the ballot identifier on every ballot position: for each ballot $b \in \{0, \ldots, \mathcal{B}-1\}$ and each within-ballot offset $k \in \{0, \ldots, \mathcal{C}-1\}$, we set $\mathsf{BallotID}(\omega^{b\mathcal{C}+k}) = b$, and we set $\mathsf{BallotID}(\omega^i) = 0$ on all padding indices $i \geq \mathcal{P}$. Next, the EA samples a secret permutation π over $\{0, \ldots, n-1\}$ and uses it to define permuted columns by $\mathsf{F}^\pi(\omega^i) = \mathsf{F}(\omega^{\pi(i)})$. The EA then commits once (pre-election or once-per-election) to the shuffled columns $\mathsf{BallotID}^\pi(X)$, $\mathsf{Code}^\pi(X)$, and $\mathsf{Mark}^\pi(X)$, and proves once that each shuffled column is a permutation of its corresponding source column under the *same* secret permutation π.

For a per-receipt query, the voter provides ballot ID b. The EA finds, privately, an index t such that $\mathsf{BallotID}^\pi(\omega^t) = b$ and $\mathsf{Mark}^\pi(\omega^t) = 1$, and it returns the receipt code $c := \mathsf{Code}^\pi(\omega^t)$ together with a batched opening proof at the single point ω^t for the three committed polynomials $\mathsf{BallotID}^\pi(X)$, $\mathsf{Mark}^\pi(X)$, and $\mathsf{Code}^\pi(X)$. Revealing the index t is safe because π is secret, so t does not reveal the candidate-order position of the mark in the original (unshuffled) ballot block.

4.3 Dispute Resolution

In a dispute, the voter provides a ballot ID b and a disputed confirmation code $c \in \mathbb{F}_q$, and the EA must prove that c does not appear anywhere in the code block for ballot b without revealing any of the ballot's codes. To do so, the EA introduces an *equality-flag* polynomial $\mathsf{Eq}(X)$ over the domain H, intended to be a binary indicator for matches against the disputed value: informally, $\mathsf{Eq}(\omega^i) = 1$ exactly when $\mathsf{Code}(\omega^i) = c$, and $\mathsf{Eq}(\omega^i) = 0$ otherwise. The EA then proves that Eq is computed consistently by also introducing an auxiliary *inverse witness* polynomial $\mathsf{Inv}(X)$ and enforcing the pointwise relation $(\mathsf{Code}(X) - c) \cdot \mathsf{Inv}(X) = 1 - \mathsf{Eq}(X)$ for all $X \in H$, together with the booleanity constraint $\mathsf{Eq}(X)(\mathsf{Eq}(X) -$

1) = 0. Intuitively, when $\mathsf{Code}(X) \neq c$, the relation forces $\mathsf{Eq}(X) = 0$ and $\mathsf{Inv}(X) = (\mathsf{Code}(X) - c)^{-1}$, while when $\mathsf{Code}(X) = c$, it allows $\mathsf{Eq}(X) = 1$ without requiring an inverse at zero.

Finally, to prove *non-membership* on ballot b, the EA uses the public ballot-block mask $\mathsf{Mask}^{(b)}_{\mathsf{ballot}}(X)$ (equal to 1 on the indices in I_b and 0 elsewhere) to isolate the relevant $\mathcal{C}$ positions. It enforces that no equality flags occur inside the ballot block by proving that the masked sum is zero:

$$\sum_{i=0}^{n-1} \mathsf{Mask}^{(b)}_{\mathsf{ballot}}(\omega^i) \cdot \mathsf{Eq}(\omega^i) = 0.$$

Equivalently, defining $\mathsf{Hit}(X) = \mathsf{Mask}^{(b)}_{\mathsf{ballot}}(X) \cdot \mathsf{Eq}(X)$, the EA proves that Hit sums to 0, which implies $\mathsf{Eq}(\omega^i) = 0$ for every $i \in I_b$ and hence that the disputed code c does not appear on ballot b.

Acknowledgements. We thank the reviewers who helped to improve our paper. J. Clark acknowledges support for this research project from (i) the National Sciences and Engineering Research Council (NSERC), Raymond Chabot Grant Thornton, and Catallaxy Industrial Research Chair in Blockchain Technologies, (ii) the AMF (Autorité des Marchés Financiers), and (iii) NSERC through a Discovery Grant.

References

1. Adida, B.: Helios: web-based open-audit voting. In: USENIX Security Symposium. pp. 335–348 (2008)
2. Adida, B., Marneffe, O.d., Pereira, O., Quisquater, J.J.: Electing a university president using open-audit voting: Analysis of real-world use of Helios. In: EVT/WOTE (2009)
3. Basin, D., Dreier, J., Giampietro, S., Radomirović, S.: Verifying table-based elections. In: ACM CCS (2021)
4. Bell, S., Benaloh, J., Byrne, M.D., DeBeauvoir, D., Eakin, B., Kortum, P., McBurnett, N., Pereira, O., Stark, P.B., Wallach, D.S., et al.: {STAR-Vote}: A secure, transparent, auditable, and reliable voting system. In: 2013 Electronic Voting Technology Workshop/Workshop on Trustworthy Elections (EVT/WOTE 13) (2013)
5. Boneh, D., Fisch, B., Gabizon, A., Williamson, Z.: A simple range proof from polynomial commitments (2019), https://hackmd.io/@dabo/B1U4kx8XI
6. Brassard, G., Chaum, D., Crépeau, C.: Minimum disclosure proofs of knowledge. J. Comput. Syst. Sci. **37**(2), 156–189 (1988)
7. Bünz, B., Fisch, B., Szepieniec, A.: Transparent snarks from dark compilers. In: CRYPTO (2020)
8. Carback, R.T., Chaum, D., Clark, J., Conway, J., Essex, A., Hernson, P.S., Mayberry, T., Popoveniuc, S., Rivest, R.L., Shen, E., Sherman, A.T., Vora, P.L.: Scantegrity II municipal election at Takoma Park: the first E2E binding governmental election with ballot privacy. In: USENIX Security Symposium (2010)

9. Chaum, D., Carback, R., Clark, J., Essex, A., Popoveniuc, S., Rivest, R.L., Ryan, P.Y.A., Shen, E., Sherman, A.T.: Scantegrity II: end-to-end verifiability for optical scan election systems using invisible ink confirmation codes. In: EVT (2008)
10. Chaum, D., Carback, R., Clark, J., Essex, A., Popoveniuc, S., Rivest, R.L., Ryan, P.Y.A., Shen, E., Sherman, A.T., Vora, P.L.: Scantegrity ii: end-to-end verifiability by voters of optical scan elections through confirmation codes. IEEE Transactions on Information Forensics and Security **4**(4), 611–627 (2009)
11. Chiesa, A., Hu, Y., Maller, M., Mishra, P., Vesely, N., Ward, N.: Marlin: Preprocessing zksnarks with universal and updatable srs. In: CRYPTO (2020)
12. Culnane, C., Ryan, P.Y., Schneider, S., Teague, V.: vvote: a verifiable voting system. ACM Transactions on Information and System Security (TISSEC) **18**(1), 1–30 (2015)
13. Essex, A., Clark, J., Hengartner, U., Adams, C.: Eperio: Mitigating technical complexity in cryptographic election verification. In: EVT/WOTE (2010)
14. Gabizon, A., Williamson, Z.J., Ciobotaru, O.: Plonk: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. Cryptology ePrint Archive (2019)
15. Goldreich, O., Micali, S., Wigderson, A.: How to prove all np statements in zero-knowledge and a methodology of cryptographic protocol design. In: CRYPTO (1986)
16. Hao, F., Ryan, P.Y.: Real-world electronic voting: Design, analysis and deployment. CRC Press (2016)
17. Kusters, R., Truderung, T., Vogt, A.: Clash attacks on the verifiability of e-voting systems. In: IEEE Symposium on Security and Privacy (2012). https://doi.org/10.1109/SP.2012.32